

Session 4: Advanced Claude Code

Skills, MCPs, and Subagents

Brady Allardice

UPF & IBEI

Where We Are

Session 1: What Claude Code can do. How it works. CLAUDE.md and plan mode.

Session 2: The core workflow. Data cleaning, estimation, robustness, $\text{\LaTeX}$ tables.

Session 3: Git and $\text{\LaTeX}$ in VS Code. Your paper joins the project.

Session 4: The layer above the core workflow — infrastructure.

Today you will:

- ➊ Migrate to a cleaner repo workflow (a structural fix, not an agent fix)
- ➋ Build a framework for deciding when a **skill** earns its keep
- ➌ Connect two **MCPs** (Zotero, GitHub) and use them in practice
- ➍ Learn when to reach for a **subagent** — and when not to

Today is about knowing which tool does real work and which is just novelty.

Since Session 3:

- Did the git cycle (commit → instruct → diff → decide) stick?
- Did anyone bring their paper into VS Code? How did it feel?
- Any tool fight back at you — compilation errors, merge conflicts, something else?

Hold onto the merge-conflict stories. We start there.

Agents solve symptoms well.
They do not notice when a problem shouldn't exist.

Fifteen minutes. One migration. The meta-lesson for the rest of the session.

The Problem We Had

Every session so far, someone has hit a merge conflict.

What we did about it:

- Deleted the local repo and recloned
- Asked Claude Code to resolve the conflict markers
- Copied work to the Desktop, wiped the directory, copied back

That worked. Everyone got unstuck. But it was a symptom fix.

The Root Cause

You were editing files *inside* the shared course repository. Every time I pushed new material, git had to reconcile your changes with mine. Conflicts were guaranteed by the structure, not by accident.

The Structural Fix

New layout on my side: each session is a self-contained folder.

```
AIAgentsCourse/  
  session_1/  
  session_2/  
  session_3/  
  session_4/ <- today's materials live here  
  ...
```

New workflow on your side: copy the session folder out of the repo into your own workspace. Work there. The shared repo stays read-only for you.

What this changes

I can push new material anytime without breaking anything in your workspace. You can edit, break, delete, and redo without worrying about the shared state. No more conflicts.

The Three-Step Ritual

Do this at the start of every session from now on:

```
# 1. Pull the latest course repo
cd ~/AIAgentsCourse && git pull

# 2. Copy today's session folder into your workspace
cp -r session_4/ ~/my_workspace/session_4/

# 3. Open your copy in VS Code and work there
cd ~/my_workspace/session_4 && code .
```

We do it together now, live. Five minutes. Everyone ends up in the same clean state before we move on.

If you had in-progress work inside the course repo, copy it into `~/my_workspace/` too. I will help you sort out what to keep.

The Meta-Lesson

Agents solve symptoms well. They do not notice when a problem shouldn't exist.

Claude Code could have resolved merge conflicts forever. It was good at it. What it would not have done is stop and say: *"Why are you editing the shared repo in the first place?"*

Your job, not the agent's: when a symptom keeps recurring, ask whether the structure is wrong.

A question to carry with you

When you catch yourself reaching for Claude Code to patch the same thing a third time, pause. Is there a structural fix you're missing?

This lesson lands again when we talk about skills (are you using it, or did you just build it?) and MCPs (does this earn a permanent connection, or is it just novelty?).

What they are.
When they earn their keep.
When they do not.

The centerpiece of today.

A skill is a reusable instruction bundle that Claude Code loads when relevant.

- Invoked by name: `/skill-name`
- Or auto-triggered when the description matches what you are doing
- Lives in `.claude/skills/<name>/SKILL.md`
- Can include helper scripts alongside the instruction file

Think of it as:

- CLAUDE.md sets *project-wide* conventions. Always on.
- A skill encodes a *specific task*. Loads when you need it.

The one-sentence definition

A skill is a named, reusable prompt with enough structure that it's worth loading on demand.

Anatomy of a Skill

A SKILL.md looks like this:

```
---
name: spec-validator
description: Validate data, variables, and transformations
  BEFORE estimation. Use when the user is about to run a
  regression, has loaded data, or mentions specification.
---

# Instructions

Before any estimation, verify:
1. Variable types match the spec ...
2. Missing data handling is explicit ...
3. Sample sizes are reported ...
[... full instructions ...]
```

The description is what earns the skill its load. Claude Code reads the description on every turn and decides whether this skill is relevant to what you're doing.

Skill vs. Prompt vs. CLAUDE.md

The key question: when is something worth being a skill?

- **Just prompt** — one-off, simple, low stakes.
Example: “Rename this variable from `x` to `income`.”
- **CLAUDE.md entry** — project-wide convention, always on.
Example: “Use pandas for data manipulation.”
- **Skill** — task-specific, recurring, carries non-trivial structure or knowledge.
Example: “Before estimation, run these 12 validation checks.”

The test question

Would a fresh prompt do this just as well? If yes, it's not a skill — it's a habit you can write down. If no, ask: what is the skill actually carrying?

A Taxonomy for Evaluating Skills

A skill earns its keep when it is doing one of these things:

- ① **Domain knowledge** — encodes facts the model would miss otherwise.
Discipline-specific conventions, clustering rules, staggered-DiD pitfalls.
- ② **Workflow structure** — enforces a multi-step procedure you'd otherwise skip steps of.
A 12-step validation protocol; a structured robustness battery.
- ③ **Critique / persona** — applies a specific lens the model wouldn't spontaneously adopt.
“Read this as a skeptical referee.” “Audit this for replication errors.”
- ④ **Output format** — produces artifacts with a rigid shape.
Codebook entries; decision-log lines; LaTeX tables in your house style.

If a skill doesn't fit one of these four boxes, it's probably a habit pretending to be infrastructure.

The Test, Concretely

For any candidate skill, ask three questions:

❶ **Would a fresh prompt produce the same output?**

If yes → not a skill.

❷ **Which taxonomy category is it in?**

Domain knowledge / workflow / critique / format. If you can't name one, be suspicious.

❸ **Will you actually invoke it?**

Skills you built and never use are dead weight. They clutter the description-matching space and make other skills worse.

The honest version

Most people build more skills than they need. The discipline isn't in building them — it's in culling them.

Demo: Critiquing My Own Skills

I'm going to walk through my actual skill list.

- Some clearly earn their keep — I will say why.
- Some are marginal — I will think out loud and let you watch me arrive at an honest answer.
- No skill gets a pass just because I built it.

This is the exercise I want you to be able to do for your own skills later.

The stance

You will build skills. Some will be great. Some will be ceremony. The difference between a useful toolkit and a bloated one is whether you're willing to kill your own creations.

My Current Skill List

From ~/.claude/skills/:

```
spec-validator/ Validate data + specs before estimation
robustness-checks/ Structured robustness battery (panel,
                    2SLS, DiD, RDD)
research-docs/ Auto-maintain README + codebook +
                decision log
auto-commit/ Install the auto-commit hook
```

Plus a few global ones (I'll mention but not dissect every one).

We'll look at two in detail: one that clearly earns its keep, and one I'm less sure about.

Earns Its Keep: spec-validator

What it does: before any regression, walks through variable types, value ranges, missingness patterns, merge diagnostics, clustering assumptions, scale mismatches.

Why it's a skill and not a prompt:

- **Domain knowledge** — encodes the dozen silent-error patterns that actually invalidate estimation (NaN coercion, wrong join keys, sample drops, improper clustering).
- **Workflow structure** — a 12-step protocol I would skip half of if I prompted ad hoc.

The fresh-prompt counterfactual:

- “Check my data before I run this regression” misses most of what matters
- A fresh prompt catches maybe 3 of the 12 checks

Verdict

Earns its keep. Two taxonomy categories. Clear value-added over a prompt.

What it claims: auto-maintain README + codebook + decision log as I work.

Walking through the test:

- *Fresh-prompt counterfactual:* could I just say “log this decision in the codebook”? Mostly yes.
- *Domain knowledge?* A little — knows what belongs in a codebook vs. a README. Not much.
- *Workflow structure?* Some — the three files stay in sync. But that’s format, not structure.
- *Will I invoke it?* Honestly... sometimes. Often I just write a note.

Thinking out loud: this is probably a *format* skill pretending to be a workflow one. It might earn its keep if I used it religiously. I don’t. So right now, it’s closer to dead weight than to infrastructure.

Honest verdict

Not sure. Leaning: demote to a CLAUDE.md paragraph and a habit. I’d rather have three

Exercise 1: Evaluate Three Skills (8 min)

Pairs. I'll put three skill descriptions on the screen. For each, decide together:

- **Keep** — it clearly fits the taxonomy and they would use it
- **Kill** — a fresh prompt would do the same work
- **Demote** — it's a CLAUDE.md line or a habit, not a skill

For each one, name:

- 1 Your verdict
- 2 Which taxonomy category (if keep) or why not (if kill/demote)

We'll share two or three answers as a group.

Disagreement is the point. If everyone agrees, I picked the wrong examples.

Exercise 2: Sketch a Skill of Your Own (10 min)

Individually, write a four-line spec for one skill you'd actually build for your own research.

- 1 Name
- 2 One-sentence description
- 3 Taxonomy category: domain knowledge / workflow / critique / format
- 4 Why a fresh prompt wouldn't do it just as well

A few will share. No coding, no installation. The deliverable is the reasoning.

The bar

If you can justify it under the test question — “would a fresh prompt do this just as well?” — you have a skill. If you can't, you have a habit. Both are fine. They just live in different places.

Skills: What to Take Away

You will leave this session with:

- A vocabulary: domain knowledge / workflow / critique / format
- A test question: *would a fresh prompt do this just as well?*
- An honest example of a marginal case, from my own toolkit
- One sketch of a skill you could build

What I did not give you

A step-by-step install guide. Building a working skill is about 15 minutes with the docs once you know you want one. Knowing whether you want one is the hard part — and that's what today was about.

Pointer to the docs: <https://docs.claude.com/en/docs/claude-code/skills>

Connecting Claude Code to systems outside your project.

Forty-five minutes. Two MCPs. One pair exercise.

What an MCP Is

MCP = Model Context Protocol. An open standard for plugging external tools into Claude Code.

- Each MCP is a separate server.
- Claude Code speaks to it over a standard protocol.
- The server exposes *tools* (like `zotero_search`) that Claude Code can call.

Without MCPs: Claude Code reads and writes files in your project. That's it.

With MCPs: it can query your Zotero library, open a GitHub PR, read a database, post to Slack — anything someone has built a server for.

The framing

MCPs turn Claude Code from a tool that operates on your *files* into a tool that operates on your *workflow*.

The Two-Reason Rule

Build or adopt an MCP for exactly one of two reasons:

- ❶ **Extend capability.** Let Claude Code do something it couldn't do at all.
Example: open a GitHub pull request. Query your Zotero library. Run a SQL query against your database.
- ❷ **Reduce friction at a boundary.** Let it do something you *could* already do, but where context-switching between tools kills the flow.
Example: inserting citations while drafting. You could look them up in Zotero and paste. But stopping to do that breaks writing flow.

Today I'm showing two specific MCPs I think are worth adopting. There are dozens more. The *framework* is what transfers; the two cases are how you learn it.

Callback to the meta-lesson: MCPs are easy to accumulate. Ask what each one earns.

The Friction Case: Zotero

Drafting with citations normally looks like:

- ① Write a sentence that needs a citation
- ② Stop. Open Zotero. Search. Find the paper. Copy the cite key.
- ③ Come back to the editor. Paste. Context-switch back into drafting.
- ④ Repeat ten times per paragraph.

With the Zotero MCP: “Find me the Smith 2019 paper on voter ID, add it to refs.bib, and cite it here.” Claude Code does the search, the BibTeX export, the `\cite{}` insertion — without you leaving the `.tex` file.

Why it earns its keep

Reduce friction at a boundary. You could already do all of this manually. The MCP just removes the tool-switch.

A single prompt:

```
Find three recent papers on voter ID laws from my  
Zotero library, add them to refs.bib, and draft one  
sentence citing each in methods.tex.
```

What you'll see in the tool-call log:

- 1 `zotero_semantic_search("voter ID laws")` — finds candidates
- 2 `zotero_get_item_metadata(...)` — pulls BibTeX fields for each
- 3 `Write refs.bib` — creates or appends to the bibliography
- 4 `Edit methods.tex` — inserts three sentences with `\cite{...}`

Watch the tool calls, not just the output. Seeing which tools fire tells you what the MCP actually does.

Where Would This Reduce Friction for You?

One minute, with the person next to you:

- Where in your own drafting workflow do you break flow to check Zotero?
- Is it frequent enough that removing the context-switch would actually matter?
- Or is it rare enough that the setup cost isn't worth it?

Installation is homework, not class time

Installing the Zotero MCP is fifteen minutes of fiddling with API keys and configuration files. I am not making you do it in class. Setup guide is in the Session 5 prework.

The Capability Case: GitHub

GitHub MCP lets Claude Code do things you couldn't do by just editing files:

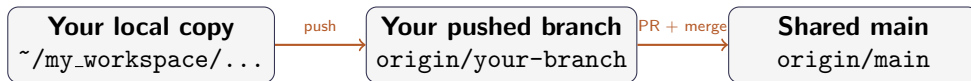
- Open a pull request from a branch you've been working on
- Leave review comments on a coauthor's PR
- Create issues to track tasks you don't want to lose
- Push files to a remote repository

Why now? This morning's Git restructuring put you in a clean state. You are working in your own workspace. You understand the three states: local copy, pushed branch, shared main. The GitHub MCP is what closes the collaboration loop.

Why it earns its keep

Extend capability. "Open a PR" isn't something Claude Code can do without this. It's a genuine new capability, not a friction fix.

The Mental Model



Branches + PRs are the collaboration mechanism. The GitHub MCP is a *helper* on top — not a substitute.

The discipline

Use `git` directly. Use the MCP when you're stuck or when the MCP genuinely saves steps. When things break, you need to understand what's actually happening — and the MCP abstracts that away.

Exercise Setup: Collaboration Pairs (5 min)

Find a partner. You will work as a pair for the next 25 minutes.

Each pair gets: a link to a small template repo with 2–3 editable files and a README explaining the exercise.

The arc:

- ① Fork the template (each of you — separate forks)
- ② Each makes a change on your own branch
- ③ Open a PR on your partner's fork
- ④ Partner reviews and merges

Use Claude Code + the GitHub MCP to do the PR opening and reviewing. Use git directly for the commits and branches.

Exercise: The PR Loop (20 min)

Pair workflow:

- 1 **Fork** the template repo I shared in chat
- 2 **Clone** your fork to `~/my_workspace/`
- 3 **Branch:** `git checkout -b my-change`
- 4 **Edit** one of the files (different files for you and your partner)
- 5 **Commit + push** the branch
- 6 **Ask Claude Code** to open a PR using the GitHub MCP:
“Open a PR from my-change to main on my fork with title X and body Y.”
- 7 **Partner reviews** the PR (also via Claude Code + GitHub MCP) and merges it

Extension, if you finish early: each edit the *same* file, hit a conflict, resolve it together.

I will circulate. Flag me when you're stuck — don't silently spin.

What to Expect

Some pairs will finish. Some will not. Both are fine.

The goal is that *everyone sees the workflow end-to-end*: branch, push, PR, review, merge. Even if you needed help at every step, you will recognize the shape next time.

What will go wrong (for some of you):

- The GitHub MCP will ask for a token you don't have. (I have a fallback.)
- The PR will open against the wrong base branch. (Fixable in the UI.)
- Your partner will merge something you weren't ready for. (That's the workflow.)

You will practice more of this in Session 5. Today: just get the shape in your head.

GitHub MCP is optional infrastructure. The underlying git workflow is not.

If you find yourself reaching for the MCP to fix a conflict — sometimes that's the right move. But sometimes (like this morning's session-folder fix) it's papering over a structural problem.

Before adopting an MCP, ask

- ❶ Does this genuinely extend what I can do, or reduce real friction?
- ❷ Will I invoke it more than three times a month?
- ❸ Am I using it because it helps, or because it's new?

Most MCPs fail question 2. The two today pass all three — *for me*. Your answers may differ.

Know they exist.

Know when to reach for them.

Don't over-invest.

Ten minutes. No hands-on.

What a Subagent Is

Claude Code can spawn another Claude Code instance inside your current session — a subagent.

- It has its own context window
- It gets a scoped task and a scoped tool set
- It returns a summary (not its raw output) to your main conversation

Two values:

- ① **Parallelism** — run three searches at once instead of serially
- ② **Context hygiene** — a big messy search doesn't pollute your main conversation

Think of it as

Delegating a bounded sub-task to a fresh colleague who doesn't see your notes — and who reports back only the conclusion.

When They Shine — and When They Don't

Subagents help when subtasks are:

- **Parallelizable** — independent work that doesn't need to be sequenced
- **Bounded** — clear inputs, clear outputs, no back-and-forth
- **Independently verifiable** — you can check the result without needing the agent's full trace

Research workflows that fit:

- Multi-database literature searches (one agent per database)
- Coding many documents against a fixed rubric
- Parallel robustness checks on independent specifications

Research workflows that don't fit:

- Anything sequential and judgment-heavy — most of your drafting, most of your analysis decisions
- Tasks that depend on your ongoing conversation context (subagents start cold)

Most research tasks do not have the shape that makes subagents worth it.

- Most of what you do is sequential
- Most of your decisions depend on context the subagent won't see
- Most of your verification needs the full trace, not a summary

If your workflow has parallel, independent, verifiable subtasks — revisit this. Otherwise, don't over-engineer.

The meta-lesson, one more time

Subagents are genuinely useful for a specific shape of problem. They're also the shiniest thing in the toolkit right now. Asking *“does my problem have that shape?”* is the whole game.

What You Built Today

- ① **A cleaner workflow.** You're working in your own workspace. No more merge conflicts against the shared repo.
- ② **A framework for skills.** Four categories, one test question. Enough to decide what earns a skill and what's just a habit.
- ③ **Two MCPs on your map.** Zotero (friction) and GitHub (capability). The two-reason rule to evaluate others.
- ④ **An honest view of subagents.** Know they exist. Know the shape of problem they solve. Don't reach for them until your problem has that shape.

The thread running through

Agents solve symptoms well. Your job is to notice when the structure is wrong, when the skill is ceremony, when the MCP is novelty, and when the subagent is overkill.

Session 5: Expert Judgment and Capstone.

- Failure patterns — what goes wrong and how to recognize it
- When to correct the approach, restart the conversation, or do it yourself
- Capstone: build an end-to-end research artifact using everything from Sessions 1–4

Before next session:

- ➊ **Install the Zotero MCP** (setup guide linked in the course README)
- ➋ **Skim your own skill list** — if you have any, run today's test on them
- ➌ **Try one thing from today** on your actual research: a skill, an MCP, or just the new repo workflow

Bring a question, a frustration, or a small win from the week. Session 5 opens with those.

